

Task 2: Job Posting with Skill Thresholds

Job-Supply Instrumentation & Validated Jobs-Posted Live View Brief

AUTHOR / CANDIDATE

Jeevan N.

ROLE

Data Analyst

FOCUS AREA

Track Job Supply

STATUS

COMPLETED & DEMOABLE

1. Executive Summary

As part of the PlaceMux Data-Analyst Marketplace Phase 2 initiative, Task 2 focuses on instrumenting job supply metrics by tracking recruiter job creation events with specific candidate skill threshold requirements. An end-to-end Python/Pandas data validation pipeline was implemented in Jupyter Notebook to process incoming raw event streams, validate payload schemas, flatten nested skill criteria, and build a live `jobs_posted_view`.

2. Key Deliverables & Achievements

✓ Job-Supply Event Instrumentation

Designed and implemented event schema tracking for `job_post_submitted` events, capturing recruiter identifiers, job role details, and target skill score thresholds.

✓ Automated Payload Validation Pipeline

Engineered schema validation logic to verify non-null key constraints and restrict skill minimum threshold scores to valid ranges (0–100).

✓ Live Jobs-Posted Aggregate View

Unnested JSON arrays into relational tabular formats, enabling immediate queryability, grouping by skill type, and reporting average demand criteria.

3. Analytical Findings & Summary Metrics

Analysis of the validated live postings view revealed key insights regarding marketplace job supply and skill expectations:

Skill Name	Total Job Postings	Avg Min Score Threshold	Marketplace Demand Level
SQL	2	85.0 / 100	High Demand / High Threshold
Python	1	75.0 / 100	Moderate Demand / Standard Threshold
Tableau	1	85.0 / 100	Specialized Demand / High Threshold

4. Implementation Code Snippet

The core pipeline executed inside Jupyter Notebook (`phase2-task2.ipynb`):

```

import pandas as pd

# 1. Event Payload Validation
def validate_job_event(row):
    payload = row['payload']
    if not payload.get('job_id') or not payload.get('skill_thresholds'):
        return False
    for skill in payload['skill_thresholds']:
        if not (0 <= skill.get('min_score', -1) <= 100):
            return False
    return True

df_events['is_valid'] = df_events.apply(validate_job_event, axis=1)
df_validated = df_events[df_events['is_valid']].copy()

# 2. Build Flattened jobs_posted_view
jobs_data = []
for _, row in df_validated.iterrows():
    p = row['payload']
    for skill in p['skill_thresholds']:
        jobs_data.append({
            'job_id': p['job_id'],
            'title': p['title'],
            'recruiter_id': p['recruiter_id'],
            'posted_at': row['timestamp'],
            'skill_name': skill['skill'],
            'min_score_threshold': skill['min_score']
        })

jobs_posted_view = pd.DataFrame(jobs_data)

```

5. Task Verification Checklist

- **Job-supply instrumentation complete:** Ingestion and structured tracking of incoming job creation payloads verified.
- **Job-post events validated:** Validation function successfully isolates invalid payloads and passes standardized event schemas.
- **Jobs-posted view live:** Real-time tabular views and dual-axis visual analytics rendered and ready for marketplace demo.